

SIMATS ENGINEERING ASSESSMENT TOOL 3: Application Based Questions

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO2: Analyze virtualization architectures to identify performance and security issues in cloud computing environments. (BL3)

Dave's Taxonomy: Manipulation (Level 2)
Question Count: 5

Total Marks: 25

Code of Conduct

I G. Ramya/192572073 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: G.Ramya

Assessment Tasks

Application Based Questions

1. A company experiences unpredictable traffic spikes during seasonal sales. Explain how cloud auto-scaling and load balancing can handle this demand efficiently. How does this approach reduce downtime and improve user experience? What role do monitoring tools play in this setup? Relate your answer to a real-world e-commerce scenario.
2. An organization wants to store large volumes of backup data securely in the cloud. Discuss how cloud storage solutions ensure durability and data redundancy. How can encryption and access control protect sensitive backup data? Explain the cost advantages compared to traditional storage systems. Provide an example of a disaster recovery use case.
3. A mobile app company plans to deploy its application globally. Explain how content delivery networks (CDNs) improve performance and reduce latency. How do edge

servers enhance user experience across regions? Discuss how cloud providers support global availability. Relate this to a real-time streaming or gaming application.

4. A financial institution requires strict compliance and data security in cloud usage. Explain how cloud providers support regulatory compliance and auditing. What role do logging, monitoring, and encryption play in security? How can the company ensure data sovereignty requirements are met? Give an example involving sensitive financial transactions.
5. A development team wants to adopt containerization for application deployment. Explain how containers differ from virtual machines in terms of performance and resource usage. How does orchestration (like Kubernetes) simplify deployment and scaling? Discuss portability benefits across different cloud environments. Provide an example of a microservices-based application.

Application Based Questions Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Relevance to Application	25%	Response directly addresses the given use case with strong relevance	Mostly relevant response	Partially relevant	Weak relevance
Use of Cloud Concepts	25%	Applies appropriate concepts accurately	Mostly accurate application	Basic application only	Incorrect concept usage
Practical Insight	20%	Shows strong real-world understanding	Shows reasonable practical understanding	Limited practical insight	No practical insight
Completeness	15%	Response covers all required aspects	Covers most aspects	Covers only basic aspects	Incomplete response
Clarity	15%	Clear and concise answer	Generally clear	Some confusion in expression	Poor clarity

1. Cloud Auto-Scaling and Load Balancing for Unpredictable Traffic

Introduction

During seasonal sales, e-commerce websites may experience sudden increases in users and transactions. Cloud **auto-scaling** and **load balancing** help applications handle these traffic spikes efficiently without affecting performance.

1. Auto-Scaling

- Auto-scaling automatically increases or decreases the number of cloud resources based on demand.
- During a sale, additional virtual machines or containers are created when CPU usage or traffic increases.
- When the traffic decreases, unnecessary resources are removed.
- This provides **rapid elasticity** and avoids resource wastage.

2. Load Balancing

- A load balancer distributes incoming user requests among multiple servers.
- It prevents a single server from becoming overloaded.
- If one server fails, traffic can be redirected to healthy servers.
- This improves application availability and reliability.

3. Reduction of Downtime

The combination works as follows:

Users → Load Balancer → Multiple Application Servers → Database

- Auto-scaling adds servers during high demand.
- The load balancer distributes traffic among them.
- Failed servers are removed from the traffic path.
- Therefore, service interruption and downtime are minimized.

4. Improved User Experience

- Faster page loading
- Reduced response time
- Fewer failed transactions
- Better handling of concurrent users

- Continuous availability during peak periods

5. Role of Monitoring Tools

Monitoring tools continuously observe:

- CPU and memory utilization
- Network traffic
- Number of active users
- Response time
- Error rates
- Server health

When predefined thresholds are reached, monitoring systems can trigger auto-scaling actions and generate alerts.

Real-World Example

Consider an e-commerce website such as Amazon during a major festive sale. Millions of users may access the website simultaneously. Cloud auto-scaling can launch additional application servers, while load balancing distributes customer requests among available servers.

Conclusion

Thus, **auto-scaling provides flexibility**, while **load balancing distributes workload efficiently**. Monitoring tools coordinate the system by detecting changes and failures. Together, these technologies provide **high availability, reduced downtime, better performance, and improved customer experience** during unpredictable traffic spikes.

2. Secure and Cost-Effective Cloud Storage for Backup Data

Introduction

Organizations generate large amounts of backup data that must be stored securely and made available during failures or disasters. Cloud storage provides **durability, redundancy, encryption, access control, and cost efficiency**.

1. Data Durability

- Cloud providers store data using highly reliable storage infrastructure.

- Multiple copies of data can be maintained.
- Data is protected against hardware failures.
- Advanced storage systems provide very high durability.

2. Data Redundancy

Cloud storage can replicate data:

- Across multiple storage devices
- Across different availability zones
- Across geographically separated regions

If one storage location becomes unavailable, another copy can be used.

3. Encryption

Encryption protects backup data from unauthorized access.

Encryption at Rest:

Protects data while it is stored.

Encryption in Transit:

Protects data while it moves between the organization and cloud storage.

Encryption ensures that stolen or intercepted data cannot easily be read.

4. Access Control

Access control mechanisms ensure that only authorized users can access backups.

Examples include:

- User authentication
- Role-Based Access Control (RBAC)
- Multi-factor authentication
- Least-privilege permissions
- Access policies

5. Cost Advantages

Compared with traditional storage:

- No need to purchase large storage hardware initially.
- No requirement for maintaining physical storage infrastructure.
- Pay-as-you-use pricing reduces unnecessary expenditure.

- Storage capacity can be increased whenever required.
- Organizations can use lower-cost storage tiers for older backups.

Disaster Recovery Example

Suppose a company's primary data center is destroyed due to a fire. Its critical databases and application backups are stored in a different cloud region. During disaster recovery, the organization retrieves the backups and restores its services in another location.

Conclusion

Cloud storage provides **durability, redundancy, security, scalability, and cost efficiency**. Encryption and access control protect sensitive backups, while geographically distributed copies enable reliable **disaster recovery and business continuity**.

3. CDN and Global Cloud Deployment for Mobile Applications

Introduction

When a mobile application is used globally, users located far from the main server may experience high latency. A **Content Delivery Network (CDN)** solves this problem by delivering content from servers located closer to users.

1. Working of CDN

A CDN consists of geographically distributed **edge servers**.

Basic architecture:

User → Nearest Edge Server → CDN → Origin Server

Frequently accessed content such as:

- Images
- Videos
- JavaScript files
- CSS files
- Game assets

can be cached at edge locations.

2. Reduction in Latency

Without a CDN, a user may need to communicate with a distant central server.

With a CDN:

- The request reaches a nearby edge server.
- Cached content is delivered locally.
- Network distance is reduced.
- Response time becomes faster.

3. Role of Edge Servers

Edge servers are located close to end users.

They:

- Cache frequently requested content
- Reduce network traffic to the origin server
- Reduce latency
- Improve application responsiveness
- Handle large numbers of simultaneous requests

4. Global Availability Through Cloud

Cloud providers support global applications using:

- Multiple geographic regions
- Availability zones
- Global load balancing
- CDN services
- Replication
- Automatic failover

If one region experiences failure, traffic can be redirected to another available region.

Real-Time Streaming Example

Consider a global video-streaming application such as Netflix.

When millions of users watch videos:

- Popular video content is cached at edge locations.
- Users receive content from nearby servers.
- This reduces buffering and latency.
- The origin infrastructure receives fewer repeated requests.

Similarly, online gaming applications can distribute game assets and updates through edge servers.

Conclusion

CDNs and edge computing significantly improve **speed, availability, and scalability**. By placing content closer to users and using multiple cloud regions, global applications can provide a **fast and reliable user experience**.

4. Regulatory Compliance and Security in Cloud Computing

Introduction

Financial institutions handle highly sensitive information such as customer details, account information, and financial transactions. Therefore, cloud deployment must satisfy strict **security, compliance, auditing, and data sovereignty requirements**.

1. Regulatory Compliance

Cloud providers support compliance through:

- Security standards and certifications
- Compliance frameworks
- Auditing mechanisms
- Data protection controls
- Access management
- Security policies

The organization must also configure its cloud environment correctly because compliance is generally a **shared responsibility**.

2. Logging

Logging records important activities such as:

- User login attempts
- Data access
- Configuration changes
- Financial transactions
- Administrative activities

Logs help identify suspicious activities and provide evidence during audits.

3. Monitoring

Continuous monitoring detects:

- Unauthorized access
- Abnormal network traffic
- Failed login attempts
- Security threats
- Unusual transactions

Alerts can be generated when suspicious behavior is detected.

4. Encryption

Encryption protects financial data from unauthorized disclosure.

Data at rest: encrypted while stored in databases and backups.

Data in transit: encrypted while transferred across networks.

Encryption keys should be securely managed and access should be restricted.

5. Data Sovereignty

Data sovereignty means data must be stored and processed according to the laws of the country or region where it is required to remain.

An organization can:

- Select appropriate cloud regions.
- Restrict data replication to approved locations.
- Apply region-based access policies.
- Monitor where sensitive information is stored.

Example: Financial Transaction

Suppose a bank processes an online money transfer. The transaction data should be encrypted, access should be limited to authorized systems, and every transaction should be logged. Monitoring systems can detect unusual transaction patterns, while audit logs provide evidence for regulatory review.

Conclusion

Cloud providers support financial institutions through **encryption, identity management, logging, monitoring, auditing, and compliance controls**. Proper region selection and data-location policies help satisfy **data sovereignty requirements**, ensuring secure and legally compliant financial operations.

5. Containerization, Kubernetes, and Cloud Portability

Introduction

Containerization packages an application along with its required libraries and dependencies into a lightweight, isolated unit called a **container**. Containers are widely used for developing and deploying modern cloud applications.

1. Containers vs Virtual Machines

Feature	Containers	Virtual Machines
Operating System	Share host OS kernel	Each VM has a guest OS
Resource Usage	Low	Higher
Startup Time	Very fast	Relatively slower
Size	Lightweight	Larger
Performance	Near-native	More overhead
Density	Many containers per host	Fewer VMs per host

Containers are therefore suitable when applications need rapid deployment and efficient resource utilization.

2. Role of Kubernetes

Kubernetes is a container orchestration platform that automates the management of containers.

It provides:

- Automatic deployment
- Container scheduling
- Scaling
- Load balancing

- Service discovery
- Health monitoring
- Self-healing
- Rolling updates

3. Simplifying Deployment and Scaling

For example, if an application suddenly receives high traffic:

Low Traffic → 3 Containers

High Traffic → Kubernetes → 10 Containers

Kubernetes can automatically increase the number of container instances based on resource requirements. When demand decreases, unnecessary containers can be removed.

4. Portability

Containers package the application and its dependencies together. Therefore, the same container image can run across:

- Developer laptops
- Testing environments
- Private clouds
- Public clouds
- Hybrid clouds

This reduces the "**works on my machine**" problem.

5. Microservices Example

Consider an online shopping application divided into microservices:

User Service | Product Service | Payment Service | Order Service | Delivery Service

Each service can run in a separate container.

For example:

- Product Service can be scaled when product searches increase.
- Payment Service can be independently secured.
- Order Service can be updated without stopping the entire application.

Conclusion

Containers are **lightweight, portable, and resource-efficient**, while Kubernetes provides automated orchestration, scaling, monitoring, and recovery. Therefore, containerization is highly suitable for **microservices and modern multi-cloud applications**.